

Build Task — Versioned Release Gate for Project 5

— Hints & Solution

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper Python). Read Basic first even if you already know Python — it's the fastest way to spot exactly what each deeper level adds. This Build Task pulls together everything from the 5 practice exercises above it — a version log, a gate, a live pointer that resists tampering, and a rollback that goes back to the last version *known to be good* — into one working system.

- [Hint 1 — What you're building, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)
- [Solution](#)

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — What you're building, and the exact pieces

Basic Version

You're building 4 small files that work together: one that saves and loads named versions of a prompt/setting (`versioning.py`), one that's the actual gate — testing a version and deciding whether it gets to go live (`deploy_gate.py`), one that undoes a bad release (`rollback.py`), and one that proves all of it actually works (`test_deploy_gate.py`).

The key idea tying it all together: "what's live right now" should never be a single fact that lives in one plain file someone could edit by hand. It should be the *answer to a question* — "looking at everything that's ever happened, what's the most recent version that actually passed?" — worked out from a running history, the same way `gate_bypass` and `rollback_trigger` already got you thinking.

Here are the exact pieces you need to look up and use:

- A `@dataclass` (from `dataclasses`) for `VersionRecord` and `DeployResult` — clean, typed containers for the data moving between these files.
- `json.dump()` / `json.load()` — same as every earlier exercise, for saving the version log and the history of deploy/rollback events.
- `datetime.now(timezone.utc).isoformat()` — for timestamps; ISO-format strings also happen to sort correctly with plain `<` and `>`, which matters for "what was live at time T."
- Your `run_eval_suite()` stand-in from earlier exercises, now taking a whole `config` dict instead of just a prompt string — real Project 5 settings are more than one string.
- `pytest` — a test function is just `def test_something(): assert some_condition;` `pytest` finds and runs every function starting with `test_` in a file.

Intermediate Version

Think of this Build Task as 3 real jobs, not 1: **storing** versions (`versioning.py`), **gating** a release (`deploy_gate.py`), and **recovering** from a bad one (`rollback.py`) — each file does one job, and each reads/writes the same underlying data, not 3 separate copies of the truth.

The design that makes "the gate can't be skipped by accident" actually true: instead of a `live_version.txt` that any text editor can rewrite with zero friction, keep an **append-only event log** — every `deploy()` call, pass or fail, appends one event; every `rollback_to_previous()` call appends one more. "What's live right now" is never stored directly — it's *derived*, every time, as "the version named in the most recent `deploy_pass` or `rollback` event." There's no separate pointer file left for someone to edit without realizing there's a gate behind it at all.

The exact pieces:

- `save_version(name: str, config: dict) -> VersionRecord` and `get_version(name: str) -> VersionRecord | None` in `versioning.py` — a thin, reusable layer other files build on, same shape as the `version_log` exercise.
- `deploy(version_name: str, threshold: float) -> DeployResult` in `deploy_gate.py` — looks up the version, runs `run_eval_suite(config)`, and appends a `deploy_pass` or `deploy_fail` event to the history depending on the score, exactly like `release_gate`'s exercise, but recording an event instead of writing a pointer file.
- `get_live_version() -> str | None` and `get_live_version_at(timestamp: str) -> str | None` — both scan the same event history; the second just ignores every event after `timestamp`.
- `rollback_to_previous() -> VersionRecord` in `rollback.py` — finds the currently live version from the history, then walks backward through earlier `deploy_pass` / `rollback` events to find the most recent *different* version, and appends a `rollback` event pointing at it. This is exactly `rollback_trigger`'s idea of "go back to the last version *known to have passed*," now wired to real version records instead of a stub.
- `pytest` basics, as in Doc13: a test file imports the functions under test, and each `test_...` function sets up a small scenario and asserts the result. Give each test its own file names (and delete them at the start), so tests never share data with each other or with your real `versions.json`.

Difference between Basic and Intermediate: Basic names the 4 files, their jobs, and the exact tools in plain words. Intermediate explains the append-only event history that answers "what's live" by working it out instead of storing it, and how `rollback_to_previous()` walks that history back to the last version that actually passed.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
versioning.py:
    VersionRecord holds: version_name, config, created_at

    function save_version(name, config):
        load existing versions list (or start empty)
        append a new VersionRecord
        save the whole list back
        return the new record

    function get_version(name):
        search the loaded list for a matching version_name
        return it, or None if not found

deploy_gate.py:
    DeployResult holds: version_name, passed, score, failing_cases

    function deploy(version_name, threshold):
        look up the version with get_version()
        if not found: raise an error
        run the eval suite against its config
        passed = score >= threshold
        record a "deploy_pass" or "deploy_fail" event in history.json,
            with a timestamp
        return a DeployResult

    function get_live_version():
        read history.json
        find the most recent event that is "deploy_pass" or "rollback"
        return its version_name (or None if there isn't one)

rollback.py:
    function rollback_to_previous():
        find the current live version from history
        walk backward through history for an earlier deploy_pass/rollback
        event with a DIFFERENT version_name
```

```

    if found: record a "rollback" event pointing at it, return that version
    if not found: raise an error

```

Here's almost the whole thing for `deploy_gate.py` — just try running it and reading it line by line:

```

import json
from datetime import datetime, timezone

def run_eval_suite(config):
    if "politely" in config.get("prompt", ""):
        return {"score": 1.0, "failed": []}
    return {"score": 0.6, "failed": ["greeting_case", "citation_case"]}

def record_event(event_type, version_name, path="history.json"):
    try:
        with open(path) as f:
            history = json.load(f)
    except FileNotFoundError:
        history = []
    history.append({
        "event": event_type,
        "version_name": version_name,
        "at": datetime.now(timezone.utc).isoformat(),
    })
    with open(path, "w") as f:
        json.dump(history, f, indent=2)

def deploy(version_name, threshold):
    from versioning import get_version
    record = get_version(version_name)
    result = run_eval_suite(record["config"])
    passed = result["score"] >= threshold
    if passed:
        record_event("deploy_pass", version_name)
    else:
        record_event("deploy_fail", version_name)
    return {
        "version_name": version_name,
        "passed": passed,
        "score": result["score"],
        "failing_cases": result["failed"],
    }

```

Expected output, given a saved version `"v1"` with `config={"prompt": "Answer fast."}`, calling `deploy("v1", 0.9)`: `{'version_name': 'v1', 'passed': False, 'score': 0.6, 'failing_cases': ['greeting_case', 'citation_case']}`.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
versioning.py:
    @dataclass VersionRecord: version_name: str, config: dict, created_at: str

    function load_versions(path) -> list[VersionRecord]
    function save_version(name, config, path) -> VersionRecord
    function get_version(name, path) -> VersionRecord | None

deploy_gate.py:
    @dataclass DeployResult: version_name: str, passed: bool,
                             score: float, failing_cases: list

    class VersionNotFoundError(Exception)

    function load_history(path) -> list[dict]
    function record_event(event_type, version_name, path) -> None
    function get_live_version(path) -> str | None
    function get_live_version_at(timestamp, path) -> str | None
    function deploy(version_name, threshold, versions_path,
                    history_path) -> DeployResult

rollback.py:
    class NoPreviousVersionError(Exception)
    function rollback_to_previous(history_path, versions_path) -> VersionRecord
```

```
# practice/build_task/versioning.py
import json
from dataclasses import dataclass, asdict
from datetime import datetime, timezone
from pathlib import Path

# why: one saved version as real fields, not a loose dict
@dataclass
class VersionRecord:
    version_name: str
    config: dict
    created_at: str
```

```

def load_versions(path: str = "versions.json") -> list[VersionRecord]:
    if not Path(path).exists():
        return []
    with open(path) as f:
        raw = json.load(f)
    records = []
    for item in raw:
        records.append(VersionRecord(
            version_name=item["version_name"],
            config=item["config"],
            created_at=item["created_at"],
        ))
    return records

def save_version(name: str, config: dict,
                path: str = "versions.json") -> VersionRecord:
    # how: load first, then append – the version_log exercise's rule
    versions = load_versions(path)
    record = VersionRecord(
        version_name=name,
        config=config,
        created_at=datetime.now(timezone.utc).isoformat(),
    )
    versions.append(record)
    as_dicts = []
    for v in versions:
        as_dicts.append(asdict(v))
    with open(path, "w") as f:
        json.dump(as_dicts, f, indent=2)
    return record

def get_version(name: str,
                path: str = "versions.json") -> VersionRecord | None:
    for v in load_versions(path):
        if v.version_name == name:
            return v
    return None

```

```

# practice/build_task/deploy_gate.py
import json
from dataclasses import dataclass
from datetime import datetime, timezone
from pathlib import Path

```

```

from versioning import get_version

class VersionNotFoundError(Exception):
    pass

# why: the gate's decision as real fields – release_gate's Approach 2
@dataclass
class DeployResult:
    version_name: str
    passed: bool
    score: float
    failing_cases: list

def run_eval_suite(config: dict) -> dict:
    # stand-in for your real Doc13 suite – swap it in here
    if "politely" in config.get("prompt", ""):
        return {"score": 1.0, "failed": []}
    return {"score": 0.6, "failed": ["greeting_case", "citation_case"]}

def load_history(path: str = "history.json") -> list[dict]:
    if not Path(path).exists():
        return []
    with open(path) as f:
        return json.load(f)

def record_event(event_type: str, version_name: str,
                 path: str = "history.json") -> None:
    # why: append-only – every gate decision stays on record, with a
    # time, so "what was live at time T" can always be answered
    history = load_history(path)
    history.append({
        "event": event_type,
        "version_name": version_name,
        "at": datetime.now(timezone.utc).isoformat(),
    })
    with open(path, "w") as f:
        json.dump(history, f, indent=2)

def get_live_version_at(timestamp: str,
                       path: str = "history.json") -> str | None:
    # how: "live" is the latest pass or rollback up to that time –

```

```

# worked out from the history, never stored in a separate file
passing = []
for event in load_history(path):
    counts = event["event"] in ("deploy_pass", "rollback")
    if counts and event["at"] <= timestamp:
        passing.append(event)
if not passing:
    return None
return passing[-1]["version_name"]

def get_live_version(path: str = "history.json") -> str | None:
    now = datetime.now(timezone.utc).isoformat()
    return get_live_version_at(now, path)

def deploy(version_name: str, threshold: float,
           versions_path: str = "versions.json",
           history_path: str = "history.json") -> DeployResult:
    record = get_version(version_name, versions_path)
    if record is None:
        raise VersionNotFoundError(f"No saved version named: {version_name}")

    result = run_eval_suite(record.config)
    passed = result["score"] >= threshold
    if passed:
        event_type = "deploy_pass"
    else:
        event_type = "deploy_fail"
    record_event(event_type, version_name, history_path)

    return DeployResult(
        version_name=version_name,
        passed=passed,
        score=result["score"],
        failing_cases=result["failed"],
    )

```

Now write `rollback.py`'s `rollback_to_previous()` yourself, using the same walk-backward-through-history idea from Hint 1, and compare against the [Solution](#).

Difference between Basic and Intermediate: Basic's pseudocode and near-complete code get the event-history idea working in one file. Intermediate turns it into the typed, multi-file design the Build Task suggests — `VersionRecord`, `DeployResult`, history helpers — with a test for each row of the Test Cases table.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Solution

Story — the Build Task: a gate the team can't skip by accident is what makes "we test before we release" true instead of hoped. This pulls the five exercises together: a version log, a gate, "live" worked out from a record instead of a file anyone can edit, and a rollback to the last version that actually passed. **If not:** Project 5's prompts could change in production with no test, no record, and no fast way back.

Every file lives in `19_mlops_llmops/practice/build_task/` and runs from inside that folder. Every code block below runs as shown. Read both depths — they're 2 real, valid ways to build the same gate, with real tradeoffs between them.

Basic Version

Approach 1 — one file, the direct way

Story: get the whole loop — save, deploy through the gate, see what's live, roll back — working in one file first. **If not:** you'd be debugging imports between four files before the idea itself worked.

```
# practice/build_task/deploy_gate.py — everything in one file for now
import json
from datetime import datetime, timezone
from pathlib import Path

def run_eval_suite(config):
    if "politely" in config.get("prompt", ""):
        return {"score": 1.0, "failed": []}
    return {"score": 0.6, "failed": ["greeting_case", "citation_case"]}

def load_versions(path="versions.json"):
    if not Path(path).exists():
        return []
    with open(path) as f:
        return json.load(f)

def save_version(name, config, path="versions.json"):
    versions = load_versions(path)
    versions.append({
        "version_name": name,
        "config": config,
```

```

        "created_at": str(datetime.now()),
    })
    with open(path, "w") as f:
        json.dump(versions, f, indent=2)

def get_version(name, path="versions.json"):
    for v in load_versions(path):
        if v["version_name"] == name:
            return v
    return None

def load_history(path="history.json"):
    if not Path(path).exists():
        return []
    with open(path) as f:
        return json.load(f)

def record_event(event_type, version_name, path="history.json"):
    history = load_history(path)
    history.append({
        "event": event_type,
        "version_name": version_name,
        "at": datetime.now(timezone.utc).isoformat(),
    })
    with open(path, "w") as f:
        json.dump(history, f, indent=2)

def get_live_version(path="history.json"):
    passing = []
    for e in load_history(path):
        if e["event"] in ("deploy_pass", "rollback"):
            passing.append(e)
    if not passing:
        return None
    return passing[-1]["version_name"]

def deploy(version_name, threshold):
    record = get_version(version_name)
    if record is None:
        print("No such version:", version_name)
        return None

    result = run_eval_suite(record["config"])
    if result["score"] >= threshold:
        record_event("deploy_pass", version_name)
        print(version_name, "deployed, score", result["score"])
    else:

```

```

        record_event("deploy_fail", version_name)
        print(version_name, "NOT deployed, score", result["score"],
              "failing:", result["failed"])
    return result

def rollback_to_previous():
    history = load_history()
    passing = []
    for e in history:
        if e["event"] in ("deploy_pass", "rollback"):
            passing.append(e)
    if not passing:
        print("Nothing has ever passed the gate.")
        return None

    current_live = passing[-1]["version_name"]
    for e in reversed(passing[:-1]):
        if e["version_name"] != current_live:
            record_event("rollback", e["version_name"])
            print("Rolled back to", e["version_name"])
            return e["version_name"]

    print("No earlier passing version to roll back to.")
    return None

save_version("v1", {"prompt": "Answer the customer's question."})
save_version("v2", {"prompt": "Answer politely and cite the source."})

deploy("v1", 0.5)
deploy("v2", 0.9)
print("live:", get_live_version())
rollback_to_previous()
print("live:", get_live_version())

```

Expected output:

```

v1 deployed, score 0.6
v2 deployed, score 1.0
live: v2
Rolled back to v1
live: v1

```

This version works correctly for every requirement and constraint in the Build Task — it's one big file instead of the suggested 4, uses plain dicts instead of dataclasses, `rollback_to_previous()` returns just the version name instead of a full `VersionRecord`, and there's no

`test_deploy_gate.py` yet. All fine for a first working version; Intermediate splits it apart properly and fixes the return type.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — split into `versioning.py` / `deploy_gate.py` / `rollback.py` , with dataclasses

Story — `versioning.py` : the version log from `version_log` , with each record as a typed `VersionRecord` . **If not:** every other file would reach into loose dicts and hope the key names matched.

```
# practice/build_task/versioning.py
import json
from dataclasses import dataclass, asdict
from datetime import datetime, timezone
from pathlib import Path

# why: one saved version as real fields, not a loose dict
@dataclass
class VersionRecord:
    version_name: str
    config: dict
    created_at: str

def load_versions(path: str = "versions.json") -> list[VersionRecord]:
    if not Path(path).exists():
        return []
    with open(path) as f:
        raw = json.load(f)
    records = []
    for item in raw:
        records.append(VersionRecord(
            version_name=item["version_name"],
            config=item["config"],
            created_at=item["created_at"],
        ))
    return records

def save_version(name: str, config: dict,
                path: str = "versions.json") -> VersionRecord:
    # how: load first, then append – the version_log exercise's rule
    versions = load_versions(path)
```

```

    record = VersionRecord(
        version_name=name,
        config=config,
        created_at=datetime.now(timezone.utc).isoformat(),
    )
    versions.append(record)
    as_dicts = []
    for v in versions:
        as_dicts.append(asdict(v))
    with open(path, "w") as f:
        json.dump(as_dicts, f, indent=2)
    return record

def get_version(name: str,
                path: str = "versions.json") -> VersionRecord | None:
    for v in load_versions(path):
        if v.version_name == name:
            return v
    return None

```

Story — `deploy_gate.py` : `release_gate` 's gate with its `DeployResult` (Approach 2), plus `gate_bypass` 's idea taken one step further: every gate decision is a timestamped event in an append-only history, and "what's live" is worked out from that history — there is no pointer file to edit. **If not:** the Build Task's "not skippable by accident" constraint would rest on nobody touching a file.

```

# practice/build_task/deploy_gate.py
import json
from dataclasses import dataclass
from datetime import datetime, timezone
from pathlib import Path

from versioning import get_version

class VersionNotFoundError(Exception):
    pass

# why: the gate's decision as real fields – release_gate's Approach 2
@dataclass
class DeployResult:
    version_name: str
    passed: bool
    score: float
    failing_cases: list

```

```

def run_eval_suite(config: dict) -> dict:
    # stand-in for your real Doc13 suite – swap it in here
    if "politely" in config.get("prompt", ""):
        return {"score": 1.0, "failed": []}
    return {"score": 0.6, "failed": ["greeting_case", "citation_case"]}

def load_history(path: str = "history.json") -> list[dict]:
    if not Path(path).exists():
        return []
    with open(path) as f:
        return json.load(f)

def record_event(event_type: str, version_name: str,
                 path: str = "history.json") -> None:
    # why: append-only – every gate decision stays on record, with a
    # time, so "what was live at time T" can always be answered
    history = load_history(path)
    history.append({
        "event": event_type,
        "version_name": version_name,
        "at": datetime.now(timezone.utc).isoformat(),
    })
    with open(path, "w") as f:
        json.dump(history, f, indent=2)

def get_live_version_at(timestamp: str,
                       path: str = "history.json") -> str | None:
    # how: "live" is the latest pass or rollback up to that time –
    # worked out from the history, never stored in a separate file
    passing = []
    for event in load_history(path):
        counts = event["event"] in ("deploy_pass", "rollback")
        if counts and event["at"] <= timestamp:
            passing.append(event)
    if not passing:
        return None
    return passing[-1]["version_name"]

def get_live_version(path: str = "history.json") -> str | None:
    now = datetime.now(timezone.utc).isoformat()
    return get_live_version_at(now, path)

```

```
def deploy(version_name: str, threshold: float,
           versions_path: str = "versions.json",
           history_path: str = "history.json") -> DeployResult:
    record = get_version(version_name, versions_path)
    if record is None:
        raise VersionNotFoundError(f"No saved version named: {version_name}")

    result = run_eval_suite(record.config)
    passed = result["score"] >= threshold
    if passed:
        event_type = "deploy_pass"
    else:
        event_type = "deploy_fail"
    record_event(event_type, version_name, history_path)

    return DeployResult(
        version_name=version_name,
        passed=passed,
        score=result["score"],
        failing_cases=result["failed"],
    )
```

Story — `rollback.py` : a rollback must go back to the last version that *passed*, not just "whatever came before" — which could be a version the gate refused. **If not:** a rollback could quietly re-release a version that was already known to be broken.

```
# practice/build_task/rollback.py
from versioning import get_version
from deploy_gate import load_history, record_event

class NoPreviousVersionError(Exception):
    pass

def rollback_to_previous(history_path: str = "history.json",
                       versions_path: str = "versions.json"):
    passing = []
    for event in load_history(history_path):
        if event["event"] in ("deploy_pass", "rollback"):
            passing.append(event)

    if not passing:
        raise NoPreviousVersionError(
```

```

        "No version has ever passed the gate – nothing to roll back to."
    )

    current_live = passing[-1]["version_name"]
    previous = None
    # why: walk back through versions that PASSED – never to one that
    # failed the gate, and never to the one that's live right now
    for event in reversed(passing[:-1]):
        if event["version_name"] != current_live:
            previous = event["version_name"]
            break

    if previous is None:
        raise NoPreviousVersionError(
            f"No earlier passing version before {current_live}."
        )

    record_event("rollback", previous, history_path)
    return get_version(previous, versions_path)

```

`rollback_to_previous()` returns a full `VersionRecord`, matching the Build Task's stated interface, so the caller gets the restored version's `config` and `created_at` too.

```

# practice/build_task/main.py – proving it works
from versioning import save_version
from deploy_gate import deploy, get_live_version
from rollback import rollback_to_previous

save_version("v1", {"prompt": "Answer the customer's question."})
save_version("v2", {"prompt": "Answer politely and cite the source."})

deploy("v1", 0.5)
result = deploy("v2", 0.9)
print("v2 passed:", result.passed, "score:", result.score)
print("live:", get_live_version())

restored = rollback_to_previous()
print("rolled back to:", restored.version_name)
print("live:", get_live_version())

```

Expected output (`python main.py` , starting with no `versions.json` or `history.json`):

```

v2 passed: True score: 1.0
live: v2

```

```

rolled back to: v1
live: v1

```

Approach 2 — `test_deploy_gate.py` , matching the Test Cases table

Story: one test per row of the Build Task's Test Cases table, in the same pytest style as Doc13, so "the gate works" is something you can re-check in one command. Each test uses its own freshly deleted files. **If not:** a later change could break the rollback or the "live at time T" answer, and nothing would say so.

```

# practice/build_task/test_deploy_gate.py
import os
import time
from datetime import datetime, timezone

import pytest

from versioning import save_version
from deploy_gate import deploy, get_live_version, get_live_version_at
from deploy_gate import VersionNotFoundError
from rollback import rollback_to_previous


def fresh_paths(name):
    # how: each test gets its own two files, deleted first, so tests
    # never share data with each other or with your real project
    versions_path = name + "_versions.json"
    history_path = name + "_history.json"
    for path in [versions_path, history_path]:
        if os.path.exists(path):
            os.remove(path)
    return versions_path, history_path


def test_version_above_bar_releases():
    versions_path, history_path = fresh_paths("above")
    save_version("v1", {"prompt": "Answer politely."}, path=versions_path)
    result = deploy("v1", 0.9, versions_path, history_path)
    assert result.passed is True
    assert get_live_version(history_path) == "v1"


def test_version_below_bar_refused():
    versions_path, history_path = fresh_paths("below")
    save_version("v1", {"prompt": "Answer fast."}, path=versions_path)
    result = deploy("v1", 0.9, versions_path, history_path)

```

```

assert result.passed is False
assert len(result.failing_cases) > 0
assert get_live_version(history_path) is None

def test_rollback_after_bad_release_got_through():
    versions_path, history_path = fresh_paths("rollback")
    save_version("v1", {"prompt": "Answer politely."}, path=versions_path)
    save_version("v2", {"prompt": "Answer fast."}, path=versions_path)
    deploy("v1", 0.9, versions_path, history_path)
    # a low bar lets the worse v2 through – the "bad release"
    deploy("v2", 0.5, versions_path, history_path)
    restored = rollback_to_previous(history_path, versions_path)
    assert restored.version_name == "v1"
    assert get_live_version(history_path) == "v1"

def test_what_was_live_at_time_t():
    versions_path, history_path = fresh_paths("time_t")
    save_version("v1", {"prompt": "Answer politely."}, path=versions_path)
    save_version("v2", {"prompt": "Answer politely, v2."}, path=versions_path)
    deploy("v1", 0.9, versions_path, history_path)
    time.sleep(0.01)
    time_t = datetime.now(timezone.utc).isoformat()
    time.sleep(0.01)
    deploy("v2", 0.9, versions_path, history_path)
    assert get_live_version_at(time_t, history_path) == "v1"
    assert get_live_version(history_path) == "v2"

def test_deploy_unknown_version_raises():
    versions_path, history_path = fresh_paths("unknown")
    with pytest.raises(VersionNotFoundError):
        deploy("does-not-exist", 0.9, versions_path, history_path)

```

Expected output (`pytest test_deploy_gate.py -v`):

```

test_deploy_gate.py::test_version_above_bar_releases PASSED
test_deploy_gate.py::test_version_below_bar_refused PASSED
test_deploy_gate.py::test_rollback_after_bad_release_got_through PASSED
test_deploy_gate.py::test_what_was_live_at_time_t PASSED
test_deploy_gate.py::test_deploy_unknown_version_raises PASSED

```

Difference from Basic: Approach 1 splits the single file into the Build Task's suggested modules, each importing only what it needs, and swaps plain dicts for typed `VersionRecord` / `DeployResult` dataclasses — the same upgrade `release_gate`'s Approach 2 made. Approach 2 adds

`test_deploy_gate.py` , with one test per row of the Test Cases table plus the unknown-version case.

Which one should you actually build? Approach 1 plus Approach 2's tests — that's every Build Task requirement and constraint, and it's what you wire into Project 5. Once more than one person can write to `history.json` , add `gate_bypass` 's `hmac` signature to each event, so a hand-added event can be detected.